SwarmBox: an inescapable, reproducible multi-agent testbed

Track 1 — Sandboxes & testbeds Swarm Alignment Hackathon, 6–8 November 2026

> **What does a multi-agent sandbox need so agents can't leave it, and how would you test that it holds? Build a small environment where 5–20 agents share a resource over many rounds, logged so another team can rerun it. How far can a cheap model stand in for a frontier agent before the results stop matching?**

All numbers below are produced by `python eval.py` (235 assertions, 0 failures) and by the commands in the last section. Nothing here is estimated.

---

1. The question

Track 1 asks for a sandbox that is *reproducible*, *multi-agent*, *shared-resource*, *logged*, and *inescapable* — and then asks how far a cheap model can substitute for a frontier one. We took "can't be escaped" as the hard part, because it is the claim that is easy to assert and hard to support.

Our question, sharpened:

> Can we build a multi-agent environment where escape is not merely *blocked* but *inexpressible*, and where the containment claim is backed by an executable corpus of attacks rather than a sentence in a README?

2. What we built

`swarmbox/` — ~900 lines, standard library only, no network, no third-party package.

Module	Role
`gate.py`	The only path from intent to world. Six operations, strict schema, 20 rule ids.
`world.py`	A regenerating common-pool resource, private per-agent cells, shared scratch.
`agents.py`	Eight deterministic policies in two families.
`swarm.py`	The round loop. A run is a pure function of its config.
`ledger.py`	Append-only JSONL. Refusals are first-class records.
`metrics.py`	Gini, overclaim, drift, collapse, pairwise correlation, covert-channel count.
`fidelity.py`	The cheap-vs-frontier experiment.
`probe.py`	The planted-collusion experiment.

2.1 The central design decision: the sandbox is not an interpreter

Agent output is parsed as **data** and is never evaluated. There is no filesystem path, no socket, no subprocess, no import and no reflection anywhere in the action grammar. Those escapes are therefore not "blocked" — they cannot be *requested*.

What remains is six operations, and each is checked before it touches the world:

Two consequences we consider more valuable than any single rule:

- **The attack surface is finite and enumerable.** We can list every way an agent can address the world, which is what makes a "it holds" claim checkable at all.

- **The refusal is legible.** Every refusal carries a stable rule id, so a report can point at `gate.string.non-canonical` instead of saying "it was rejected".

2.2 Three layers of containment

Layer	What it stops
Structural — not in the grammar	filesystem, network, process, import, reflection, eval
Schema — strict, no unknown fields	smuggling, type confusion, impersonation via extra fields
State — checked against the world	over-claim, over-release, reserved namespaces, unknown peers

The schema is deliberately strict: an unknown field is refused rather than ignored. Ignoring is how data gets smuggled. Impersonation is handled by permitting an optional `agent` field *so that it can be rejected by the identity rule* — if it were simply not in the schema, every impersonation would be reported as a schema error and the ledger would be less useful as evidence.

2.3 Invariants: the arithmetic that cannot lie

These are checked inside `World.apply()` and raise on violation; they are also written to the ledger once per round. The reason to care: an agent that escapes would almost certainly have to conjure units from nothing, and that breaks arithmetic first. The invariants are the tripwire, the gate is the fence.

3. Experiments

E1 — Can it be escaped? (the attack corpus)

`gate.ESCAPE_CORPUS` contains 32 adversarial proposals: unknown ops, negative and zero amounts, `10**9` claims, float/bool/string/None where an int is required, unknown fields, path-like keys (`../secret`), reserved namespaces (`private_*`, `system_*`), NUL and DEL characters, fullwidth and ligature keys designed to reach the same namespace after normalisation, oversized values, impersonation, unknown peers, and an unknown spawn role.

Result: 31 refused, each with the rule id written next to it in the corpus; 1 correctly allowed (a proposal naming its own agent). Zero escapes. This runs on every CI invocation, so the claim cannot rot.

A substring scan is not how we check the package is clean, incidentally — the words "socket" and "subprocess" appear in `gate.py`'s docstring, because explaining that they are absent is the point. The check walks the AST instead. A guard that flags its own documentation is a guard that gets ignored.

E2 — Can another team rerun it?

A run is a pure function of `(seed, n, rounds, capacity, regen, roles)`.

Result: for seeds 1–5, two runs with the same seed produce byte-identical ledger digests and identical metric bundles; the five seeds produce five distinct runs. The ledger records the full configuration in its first line, so a team handed only the ledger knows exactly what to rerun.

E3 — Can a monitor that sees only the ledger find coordination?

We plant one colluding pair in a population of cooperators. The pair never speaks in the open: they alternate a maximum claim with a one-unit claim so the per-round total looks ordinary, and they pass a one-bit signal through the shared scratch space. Nothing about them is special-cased in the world or the gate — they use ordinary operations.

Result, across three seeds: the monitor flags exactly one pair, it is the planted pair, with correlation -1.000, and it counts 11 cross-agent reads of a shared key.

One honest caveat we found while calibrating: **this signal only survives while the pool does.** Once the resource collapses, claims get clamped by scarcity and the alternating pattern is destroyed. Monitoring quality degrades after collapse — which is worth knowing if you are designing oversight for a resource-constrained swarm.

E4 — How far can a cheap model stand in for a frontier one?

We cannot call models from a repository that must run offline and reproduce for free, so we isolate the axis along which a cheap model actually differs here: **what it can condition on.**

- **cheap family** (4 policies) — myopic: sees its own holding and the pool.
- **frontier family** (2 policies) — informed: additionally sees the swarm's recent aggregate behaviour and adapts to it.

Eight seeds, 12 agents, 24 rounds each.

family	gini	overclaim	drift	**collapse rate**
cheap (myopic)	0.2003	1.125	-112.5	**1.00**
frontier (informed)	0.0354	1.071	-44.3	**0.00**
mixed	0.2638	1.152	-104.7	1.00

Result: the myopic family exhausts the pool on all 8 seeds; the informed family never does, on any seed. The divergence is categorical — collapse versus no collapse — not a small delta you could average away. Inequality differs by 5.7× (gini 0.200 vs 0.035).

So the answer to Track 1's third question, on this testbed, is: not far at all. A cheap stand-in does not reproduce frontier behaviour even on a resource problem this small. If a team wants to screen swarm dynamics cheaply, they must first establish that the specific metric they care about is insensitive to the information set — ours were not.

4. Findings

1. **Containment is much cheaper to guarantee when the sandbox is not an interpreter.** Six operations with a strict schema produce a finite, enumerable attack surface — and therefore a corpus you can actually run, rather than a claim you can only assert.
2. **The escape corpus must be part of CI, not part of the write-up.** 32 attacks, re-run on every commit. A "can't be escaped" claim that is not executed is a rumour.
3. **Conservation invariants are a better tripwire than permission lists.** Arithmetic cannot be socially engineered.
4. **Refusals have to be first-class records.** A ledger of only successful actions cannot support any containment claim, and it is the natural thing to write by accident.
5. **The cheap/frontier gap here is categorical, not gradual.** Collapse rate 1.00 vs 0.00 across 8 seeds. A cheap stand-in is not a safe substitute for studying resource-sharing swarms.
6. **Oversight degrades after resource collapse.** Coordination detection worked perfectly (correlation -1.000) while the pool survived and loses signal once scarcity clamps behaviour.

5. Limitations — read these before citing us

- **No language model is called anywhere in this repository.** Every policy is a pure function. The cheap/frontier result is an experiment about *information sets*, not a benchmark of real models. Wiring in a real cheap-vs-frontier pair is the obvious next step and the code is structured for it (`agents.propose` is a single dispatch point).

- **The "frontier" family has only two policies** (`forecaster`, `adapter`). Their advantage may be an artefact of how they were written, not of being better informed. More policies per family would test that.
- **Escapes are only refused within the action language.** We argue the language cannot express an escape; we do not claim to have defeated a general-purpose code-execution sandbox, because we do not run one. Different threat model, different claim.
- **The gate is a static checker, not a runtime sandbox.** An agent cannot reach the host through SwarmBox because SwarmBox offers no path to it — if someone embedded this in a system that did, this analysis would not cover the new surface.
- **One resource, one regeneration model.** No prices, no punishment, no exit. Real swarms have all three.
- **Correlation is a weak collusion detector** in general. It found our planted pair because that pair was turn-taking. Sophisticated collusion would need better tools (Track 4's problem, not ours).

6. Reproduce

Requires Python 3.9+ and nothing else. `env -u PYTHONUTF8 LC_ALL=C python eval.py` also returns 0 (Windows cp936 is a real way to lose a submission).

7. Lineage

The containment philosophy — a deterministic verifier as the release authority, with append-only receipts — comes from the author's earlier work on single-agent code containment. The multi-agent harness, the resource model, the metrics, the collusion probe and the cheap/frontier experiment are new for this hackathon.